

Code:

```
#include <stdio.h>
#include <string.h>

#define MAXN 1005
#define MAXE 4005

int head[MAXN], to[MAXE], cap[MAXE], nxt[MAXE], edge_cnt = 1;
int level[MAXN], q[MAXN];

void link(int u, int v) {
    to[++edge_cnt] = v; cap[edge_cnt] = 1; nxt[edge_cnt] = head[u]; head[u] = edge_cnt;
    to[++edge_cnt] = u; cap[edge_cnt] = 0; nxt[edge_cnt] = head[v]; head[v] = edge_cnt;
}

int bfs(int n) {
    memset(level, -1, sizeof(int) * (n + 1));
    int front = 0, back = 0;
    q[back++] = 0;
    level[0] = 0;

    while (front < back) {
        int u = q[front++];
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
            if (cap[e] > 0 && level[v] == -1) {
                level[v] = level[u] + 1;
                q[back++] = v;
            }
        }
    }
    return level[n] != -1;
}

int dfs(int u, int sink, int flow) {
    if (u == sink || flow == 0) return flow;
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (level[u] + 1 == level[v] && cap[e] > 0) {
            int pushed = dfs(v, sink, 1);
            if (pushed) {
```

```

        cap[e] -= pushed;
        cap[e ^ 1] += pushed;
        return pushed;
    }
}
}
return 0;
}

int main() {
    int n, m, k;
    if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;

    int source = 0;
    int sink = n + m + 1;

    for (int i = 1; i <= n; ++i) link(source, i);
    for (int i = 1; i <= m; ++i) link(n + i, sink);

    for (int i = 0; i < k; ++i) {
        int b, g;
        scanf("%d %d", &b, &g);
        link(b, n + g);
    }

    int max_match = 0;
    while (bfs(sink)) {
        while (1) {
            int pushed = dfs(source, sink, 1);
            if (!pushed) break;
            max_match += pushed;
        }
    }

    printf("%d\n", max_match);
    for (int u = 1; u <= n; ++u) {
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
            if (v > n && v <= n + m && cap[e] == 0) {
                printf("%d %d\n", u, v - n);
            }
        }
    }
}

```

```
    return 0;  
}
```

Input:

```
3 2 4  
1 1  
1 2  
2 1  
3 1
```

Output:

```
2  
1 2  
2 1
```